

Week 1, Days 1-4

Attention, RoPE, GQA, and the GPU that runs them. Short version, built on the measured numbers in this repo: Qwen2.5-1.5B, fp16, one RTX 3060 12GB.

How to read this. Four days, one section each. Every idea gets one small worked example with real numbers you can redo by hand. Every claim ends in a number from this repo where one exists. The goal is that you can say each idea out loud in under 90 seconds without notes.

Day	Topic	The one thing you must be able to derive
1	Attention from scratch	Why the score is divided by the square root of d
2	RoPE and GQA	Why GQA shrinks the KV cache but not the weights
3	The GPU in three numbers	The ridge point: 51 TFLOPS / 360 GB/s = 142 FLOP/byte
4	Prefill vs decode	Why decode is memory-bound, in your own measured tok/s

The model and the card, once, for the whole document

Model	Qwen2.5-1.5B, fp16	GPU	RTX 3060 12GB, Ampere sm_86
params	1,543,714,304	VRAM	12,288 MiB
layers	28	SMS	28
d_model	1536	fp16 peak	~51 TFLOPS (tensor cores)
query heads	12	bandwidth	360 GB/s theoretical
KV heads	2 (GQA)		291.5 GB/s measured achievable
head_dim	128	weights on card	2945 MiB
vocab	151,936		

Day 1 · Attention, from the ground up

1.1 What the model actually does

A language model takes a sequence of tokens and produces a probability for every possible next token. You pick one, append it, and run the whole thing again. That loop has two consequences that drive everything else in this field:

1. Generation is **sequential**. Token 5 cannot start before token 4 exists. No amount of GPU fixes this.
2. Each step runs over a sequence that is one token longer than the last. Recomputing the whole sequence every step would be quadratic waste, so you cache the per-token intermediate values. That cache is the **KV cache**, and it is the single biggest memory object in serving.

Two phases follow from this, and they behave completely differently:

- **Prefill**: process the whole prompt at once. Many tokens, one pass. Big matrix work.
- **Decode**: produce one token at a time. One token, many passes. Tiny matrix work, huge memory reads.

1.2 Tokens and embeddings

Text is split into subword pieces by a BPE tokenizer. Rule of thumb: about 4 characters per token in English. A token is just an integer index, nothing more. The first layer is a lookup table of shape [vocab, d_model]: row *i* is the vector for token *i*.

EXAMPLE 1-A: THE EMBEDDING TABLE COSTS REAL MEMORY AND ZERO MATHS

151,936 vocab x 1536 d_model = 233,373,696 params
x 2 bytes (fp16) = 466,747,392 bytes = 445 MiB
share of the model = 233.4M / 1543.7M = 15.1%

15% of this model is a table you only ever *read one row from*. No multiply, no add, pure memory traffic. That is the theme of the whole field appearing on page one.

1.3 The dot product, and why attention uses it

$a \cdot b = |a||b|\cos\theta$. For two unit vectors the dot product is the cosine of the angle between them. Same direction gives +1, perpendicular gives 0, opposite gives -1. So a dot product is a cheap numeric answer to "how much do these two vectors agree?" Attention needs exactly that: how relevant is each earlier token to the one I am processing now.

1.4 Q, K, V

Each token vector is projected three times by learned weight matrices:

- **Query (Q)**: what this token is looking for.
- **Key (K)**: what this token offers to be matched against.
- **Value (V)**: what this token actually passes along if it is selected.

You compare every query against every key with dot products, turn those scores into weights that sum to 1, and take that weighted mix of the values.

```
Attention(Q, K, V) = softmax( Q KT / sqrt(d_head) ) V
```

EXAMPLE 1-B: FULL ATTENTION BY HAND, 3 TOKENS, D = 2

K = $\begin{bmatrix} 1,0 \\ 0,1 \\ 1,1 \end{bmatrix}$ V = $\begin{bmatrix} 1,0 \\ 0,1 \\ 1,1 \end{bmatrix}$ q3 = [1,1] (the third token's query)

raw scores	q3·k1 = 1	q3·k2 = 1	q3·k3 = 2	
scale /sqrt(2)	0.7071	0.7071	1.4142	
exp	2.0281	2.0281	4.1133	sum = 8.1695
weights	0.2483	0.2483	0.5035	sum = 1.000

output = 0.2483*[1,0] + 0.2483*[0,1] + 0.5035*[1,1] = [0.7517, 0.7517]

Token 3 matches itself best (score 2 against 1), so it keeps half its weight on itself and splits the rest. That is the entire mechanism.

1.5 Why divide by the square root of d

This is the derivation that gets asked. Take two vectors of length d whose entries are independent with mean 0 and variance 1. Their dot product is a sum of d such products. Variances add, so the dot product has variance d and standard deviation $\sqrt{d}$. Scores therefore grow with the head dimension. Softmax of large-spread scores saturates: it puts almost all weight on one token and the gradients through the others go to zero. Dividing by $\sqrt{d}$ puts the variance back to 1 regardless of head size.

EXAMPLE 1-C: WHAT SATURATION LOOKS LIKE AT THIS MODEL'S HEAD_DIM = 128

head_dim = 128 -> score std = $\sqrt{128} = 11.3$

unscaled, two typical scores 11.3 apart:
 $\exp(11.3) / \exp(0) = 80,822$ one token takes ~99.999% of the weight

after dividing by $\sqrt{128}$, the same pair is 1.0 apart:
 $\exp(1.0) / \exp(0) = 2.72$ a usable 73% / 27% split

1.6 Softmax numerical stability

Softmax uses $\exp$, which overflows fast. fp16 maxes out at 65,504, and $\exp(12)$ already exceeds that. The fix is that softmax is unchanged if you subtract any constant from every score, so you subtract the row maximum. Every exponent becomes 0 or negative, so every $\exp$ lands in (0, 1].

EXAMPLE 1-D: MAX SUBTRACTION

```
scores      [100, 101, 102]      exp(102) overflows fp16 and fp32 alike
minus max   [-2, -1,  0]        exp -> 0.1353, 0.3679, 1.0000  sum = 1.5032
softmax     [0.0900, 0.2447, 0.6652]  same answer, no overflow
```

Forward reference. Holding a running maximum and rescaling earlier terms when a bigger one shows up is exactly the "online softmax" trick. FlashAttention is built on it, which is why attention can be computed in tiles that fit in on-chip memory without ever forming the full score matrix. Week 2 Days 3-4.

1.7 Causal masking

Token 3 must not see token 5. Before the softmax, set every score at a future position to negative infinity.

$\exp(-\infty) = 0$, so those tokens get exactly zero weight and the remaining weights still sum to 1. It is a mask on the score matrix, not a change to the maths.

1.8 Multi-head attention, and why it is free

Instead of one attention over 1536 dimensions, split into 12 heads of 128 each, run attention independently in each, then concatenate back to 1536 and pass through an output projection.

```
12 heads x 128 head_dim = 1536 = d_model
```

The parameter count and the FLOP count are identical to single-head. You get several independent relational patterns for free. Two consequences that matter later:

- Smaller heads mean a smaller $\sqrt{d}$, so scores are better conditioned.
- A per-head score matrix is small enough to tile into on-chip SRAM. FlashAttention depends on this.

1.9 The rest of the block, briefly

- **RMSNorm**: divide the vector by its own root-mean-square, then scale by a learned gain. Cheaper than LayerNorm because it skips the mean subtraction, and models train fine without it.
- **SwiGLU MLP**: three matrices instead of two, one gating the other: $(\text{silu}(x \cdot W_{\text{gate}}) * (x \cdot W_{\text{up}})) \cdot W_{\text{down}}$. Needed because stacking linear layers with no nonlinearity in between collapses to a single linear layer.
- **Residual connections**: every block adds its output back to its input, so information and gradients have an unobstructed path from input to output.

The division of labour, in one line. Attention moves information *between* tokens. The MLP processes each token on its own. Most of the weights are MLP, so most of the memory traffic is MLP. Almost all the systems complexity is attention, because attention is the part whose cost grows with sequence length.

Sources: Vaswani et al., Attention Is All You Need · Karpathy, Let's build GPT from scratch · nanoGPT · 3Blue1Brown, Attention visually explained · Milakov & Gimelshein, online softmax

Day 2 · RoPE and the KV cache (MHA, MQA, GQA)

2.1 The problem RoPE solves

Attention is a weighted sum. Shuffle the input tokens and you get the same output in shuffled order. The mechanism has no notion of order at all, so position has to be injected deliberately.

Approach	How	Weakness
Learned absolute	A trained vector per position, added to the embedding	Cannot go past the trained maximum length. Position 5000 was never seen.
Sinusoidal absolute	Fixed sine and cosine of position, added in	Extrapolates in principle, but the model has to learn to subtract absolute positions to get relative ones.
RoPE	Rotate Q and K by an angle proportional to position	Needs interpolation tricks (YaRN, NTK scaling) far past the trained length, but is otherwise the current default.

2.2 The RoPE mechanism

Split each head's 128 dimensions into 64 pairs. Treat each pair as a point in a plane. For a token at position m , rotate pair i by the angle $m \cdot \text{theta}_i$, where $\text{theta}_i = 10000^{(-2i/\text{head_dim})}$. Early pairs rotate fast and encode fine local position; late pairs rotate slowly and encode coarse long-range position. Apply the same rotation rule to Q and to K. Nothing is added to V, and no parameters are learned.

The property that makes it work: rotations compose. Rotating Q by m and K by n , then taking the dot product, gives a result that only depends on $m - n$. The model gets relative position for free out of an absolute operation.

EXAMPLE 2-A: VERIFY THE RELATIVE PROPERTY NUMERICALLY (ONE PAIR, THETA = 1)

```
q = [1, 0] at position m = 5      k = [0, 1] at position n = 3

rotate q by 5 rad: [cos5, sin5] = [ 0.28366, -0.95892]
rotate k by 3 rad: [-sin3, cos3] = [-0.14112, -0.98999]
dot product: 0.28366*(-0.14112) + (-0.95892)*(-0.98999)
              = -0.04003 + 0.94933 = 0.90930

now the claim: this should equal q rotated by (n - m) = -2, dotted with k unrotated
rotate k by -2 rad: [-sin(-2), cos(-2)] = [0.90930, -0.41615]
q · that = 1*0.90930 + 0*(-0.41615) = 0.90930      identical
```

Positions 5 and 3 give the same score as positions 100 and 98. Only the distance survives.

2.3 The KV cache size formula

During decode you keep every past K and V so you never recompute them. The size is:

KV bytes = 2 (K and V) x layers x kv_heads x head_dim x bytes_per_element x seq_len x batch

EXAMPLE 2-B: BYTES PER TOKEN FOR THIS MODEL

2 x 28 layers x 2 kv_heads x 128 head_dim x 2 bytes = 28,672 bytes = 28 KB per token

Note which head count is used: **2 KV heads, not 12 query heads**. That single substitution is the whole GQA result.

2.4 MHA, MQA, GQA

Scheme	KV heads	Bytes / token	vs MHA	Trade
MHA (all heads have their own K,V)	12	172,032 B = 168 KB	1x	Best quality, worst memory
GQA (groups share K,V) — this model	2	28,672 B = 28 KB	6x smaller	Near-MHA quality
MQA (all heads share one K,V)	1	14,336 B = 14 KB	12x smaller	Measurable quality loss

Why GQA shrinks the cache but not the weights. GQA reduces the number of *K* and *V* heads. The 12 query heads stay, the output projection stays, and the whole MLP, which is most of the parameters, is untouched. The K and V projection matrices do shrink, but they are a small slice of the model. So the weights barely move while the per-token cache drops 6x. The KV cache grows with sequence length and batch size, the weights do not, so shrinking the growing term is what matters for serving.

2.5 The measured consequence: the OOM cliff

The experiment in this repo grew the context one token at a time on the HuggingFace path until the card died. The full write-up is docs/oom-results.md.

Quantity	Value
Context length at OOM	123,565 tokens
Weights resident	2945 MiB
Peak allocated at OOM	10,437 MiB
Peak reserved at OOM	11,764 MiB (card total 12,288 MiB)
Predicted slope (formula above)	28 KiB / token
Measured slope	62 KiB / token, 2.21x the prediction

Slope from the CSV directly: (10,437.33 - 2945.29) MiB x 1024 / 123,533 tokens = 62.10 KiB/token.

The 2x is not noise, and finding its cause is the interesting part. HuggingFace grows the cache with torch.cat([old_cache, new_kv]) on every single token. A tensor cannot grow in place, so cat allocates a fresh contiguous block for n+1 tokens and copies the old n across. For the duration of the copy, both caches are live.

EXAMPLE 2-C: THE 2X KV MODEL, CHECKED AT 120K TOKENS

analytical KV28,672 x 120,000= 3.28 GiB = 3281 MiB
weights + 1 x KV2945 + 3281= 6226 MiB (textbook answer, wrong)
weights + 2 x KV2945 + 2 x 3281= 9507 MiB (reallocation model)
measured allocated= 10,226 MiB (+~700 MiB attention scratch)

slope check: d/dn (weights + 2 KV) = 2 x 28 = 56 KiB/token vs 62 measured
the extra 6 KiB is attention working memory, which also grows with n

Two more results worth carrying into an interview:

- **The crash is fragmentation, not exhaustion.** At OOM, 10,437 MiB was allocated but 11,764 MiB was reserved. About 1.3 GiB was free inside the pool and could not be handed out, because the next cat needed one contiguous slab and the free space was scattered in gaps between parked blocks.
- **Decode slows 10x as context grows**, from ~30 tok/s to ~3 tok/s. Three costs grow with context: reading the whole cache for attention (28 KB x n), the cat copy (about 2 x 28 KB x n), and nothing else. Past roughly 10k tokens these overtake the fixed 3 GB weight read.

This is exactly what PagedAttention removes. vLLM appends new tokens into pre-allocated fixed-size blocks. No cat, no second copy, no contiguity requirement, so no fragmentation wall. Measured on the same card: vLLM held 237,376 tokens of real KV in 6.34 GiB. The HuggingFace path effectively pays for every token twice.

Sources: Su et al., RoFormer (RoPE) · EleutherAI, Rotary Embeddings: A Relative Revolution · Ainslie et al., GQA · Shazeer, Fast Transformer Decoding (MQA) · Kwon et al., PagedAttention / vLLM

Day 3 · The GPU, in the terms that decide performance

3.1 Three numbers describe a GPU

Property	RTX 3060 12GB	What it limits
Compute throughput	~51 TFLOPS fp16 on tensor cores	How fast you can do arithmetic
Memory bandwidth	360 GB/s theoretical, 291.5 GB/s measured (81%)	How fast you can feed the arithmetic
Memory capacity	12,288 MiB	What fits at all

The measured 291.5 GB/s comes from `scripts/measure_bandwidth.py` on this box, a large streaming read. Always compare against the achievable figure, not the spec sheet, or every efficiency number you quote is 19% too pessimistic.

3.2 How the chip is organised

- **SM (Streaming Multiprocessor)**: the independent unit of the GPU. This card has 28 of them. Each has its own registers, its own on-chip shared memory, and its own schedulers.
- **Thread**: one lane of execution. You launch thousands.
- **Warp**: 32 threads that execute the same instruction at the same time, always. This is the real unit of scheduling. If threads inside one warp take different branches, the warp runs both sides one after the other with half the lanes idle. That is **warp divergence**.
- **Block**: a group of warps assigned to one SM. Threads in a block can share data through that SM's shared memory.
- **Tensor core**: dedicated hardware that does a small matrix multiply-accumulate in one instruction. This is where the 51 TFLOPS lives. Ordinary FP32 arithmetic on this card is roughly 13 TFLOPS, about 4x slower. If your kernel does not hit tensor cores, you are not on the compute ceiling you think you are.

3.3 The memory hierarchy

	size (per SM unless noted)	relative latency
registers	256 KB	~1 cycle
shared memory / L1	up to 128 KB	~30 cycles
L2 cache	3 MB (whole chip)	~200 cycles
global memory (VRAM)	12,288 MiB	~400-600 cycles

Every step down is roughly an order of magnitude more capacity and an order of magnitude more latency. Optimising GPU code is almost always the act of moving a working set one level up this table and reusing it there.

EXAMPLE 3-A: WHY 3 MB OF L2 DOES NOT SAVE YOU

```
weights that must be read for one decode step = 3.09 GB
L2 cache                                     = 0.003 GB

3.09 / 0.003 = ~1000x too big to cache
```

Every decode step reads the full weight set from VRAM. There is no cache trick available. The only escape is to make that one read serve more work, which is what batching does.

3.4 Coalescing

The memory system does not serve individual addresses. It serves fixed transactions, typically 32 bytes. When the 32 threads of a warp read 32 consecutive 4-byte floats, that is 128 contiguous bytes and the hardware serves it in 4 transactions. If those same 32 threads read 32 addresses scattered across VRAM, it takes 32 transactions and you have thrown away most of the bandwidth you paid for.

EXAMPLE 3-B: COALESCED VS STRIDED, SAME 32 VALUES

```
coalesced    thread i reads addr[i]          128 bytes wanted, 128 fetched  100% useful
strided      thread i reads addr[i * 64]      128 bytes wanted, 1024 fetched 12.5% useful

on this card: 291.5 GB/s achievable -> ~36 GB/s effective, an 8x loss
```

This is why memory layout is a first-class concern, and why the KV cache layout inside a serving engine is chosen so that the attention kernel reads it contiguously.

3.5 Occupancy and latency hiding

A VRAM read takes 400 to 600 cycles. The GPU does not try to make that faster. It hides it: when a warp stalls on memory, the SM instantly switches to another resident warp. With enough resident warps there is always something to run and the stall never shows.

Occupancy is the fraction of the maximum resident warps per SM that your kernel actually achieves. It is limited by registers per thread and shared memory per block: use too much of either and fewer blocks fit on an SM at once.

The correction that matters: high occupancy is not the goal, hiding latency is. A kernel at 25% occupancy with plenty of independent work in flight can beat a kernel at 100% occupancy that stalls. Occupancy is a means, not a metric to maximise.

3.6 Arithmetic intensity and the ridge point

This is the most important idea in the whole week. For any piece of work, define:

```
arithmetic intensity = FLOPs performed / bytes moved      (unit: FLOP/byte)
```

The hardware has a break-even intensity where the compute ceiling and the bandwidth ceiling cross. That is the **ridge point**.

EXAMPLE 3-C: THE RIDGE POINT OF THIS CARD

```
51e12 FLOP/s / 360e9 byte/s = 142 FLOP/byte

below 142 -> memory-bound. You finish the maths and wait for data. Faster arithmetic
              changes nothing. Move fewer bytes.
above 142 -> compute-bound. Data arrives faster than you can use it. Better kernels and
              tensor cores help.
```

Every optimisation in LLM serving is a statement about which side of 142 you are on. Day 4 places prefill and decode on that axis with measured numbers.

Sources: [CUDA C++ Programming Guide](#) · [Boehm, How to optimize a CUDA matmul kernel](#) · [Horace He, Making Deep Learning Go Brrrr From First Principles](#) · [NVIDIA, GPU Performance Background](#)

Day 4 · Prefill vs decode, on the roofline, with measured numbers

4.1 The two phases are different shapes of matrix multiply

	Prefill	Decode
Tokens per pass	512 (the prompt)	1
Operation	GEMM, matrix x matrix	GEMV, vector x matrix
Weight bytes read	3.09 GB, once, for 512 tokens	3.09 GB, once, per token
Arithmetic intensity	hundreds of FLOP/byte	about 1-2 FLOP/byte
Side of the 142 ridge	compute-bound	far into memory-bound

The same weights, the same kernels, and a thousandfold difference in efficiency. That is the central fact of inference optimisation. Decode reads the entire model to produce one token.

4.2 Measured prefill: compute-bound, at 25% of the ceiling

EXAMPLE 4-A: PREFILL AGAINST THE COMPUTE CEILING

```
FLOPs for a forward pass ~= 2 x params x tokens
= 2 x 1.544e9 x 512      = 1.58e12 FLOP
floor at 51 TFLOPS      = 1.58e12 / 51e12 = 31 ms
measured (HuggingFace)  = 122 ms = 4205 tok/s -> 12.99 TFLOPS, 25.5% of peak
measured (vLLM)         = 124 ms = 4114 tok/s -> 25.1% of peak, 2% SLOWER
```

The "2 x params" rule comes from every weight being touched once, with one multiply and one add per weight. It counts linear layers only and ignores the attention term that grows as seq^2 , which is fine at 512 tokens (about 3% extra) and wrong past a few thousand.

Read the vLLM number carefully. vLLM does not beat HuggingFace at prefill at all here, it is 2% slower, and that is the expected answer rather than a disappointment. Both hand the same GEMMs to the same cuBLAS tensor-core kernels. There is no algorithm to beat. Prefill is already near the practical ceiling, so this is where a fast engine has the least to offer. Saying this confidently is a better signal than claiming a win that the measurement does not support.

4.3 Measured decode: memory-bound, and the number to quote is MBU

For a memory-bound phase, the right efficiency metric is not FLOPS but **memory bandwidth utilisation**: the bytes you actually moved per second, divided by the bytes the card can move.

EXAMPLE 4-B: DECODE AGAINST THE BANDWIDTH CEILING

```
bytes that must move per decode token = 3.09 GB (the weights)

vLLM  79.9 tok/s -> 12.5 ms/token -> 3.09 GB / 0.0125 s = 247 GB/s -> 85% of 291.5 achievable
HF    28.1 tok/s -> 35.6 ms/token -> 3.09 GB / 0.0356 s = 87 GB/s -> 30% of achievable

gap = 79.9 / 28.1 = 2.84x (older notes in this repo say ~4x, from two early
                           HF runs at 18.0 and 23.2 tok/s; the settled value is 28.1)

theoretical ceiling: 291.5 GB/s / 3.09 GB = 94 tok/s, whatever software you write
```

The sentence to be able to say. vLLM is at 85% of this card's achievable read bandwidth at batch 1. It is close to the hardware limit, so no further kernel work will help at batch 1. The only ways past 94 tok/s are to move fewer bytes (quantisation) or to serve more tokens per byte read (batching).

4.4 So where did HuggingFace lose the other 52 tok/s?

The obvious guess is slow kernels. The measurement says otherwise. Both engines were profiled per decode step (docs/profiling.md), separating time the GPU spent computing from time it spent idle waiting for the

CPU to hand it the next kernel. The profiling run is its own paired measurement, so its HF step time (40.65 ms, 24.6 tok/s) differs slightly from the 28.1 tok/s baseline run. Compare rows within this table, not against the table above.

Run	GPU kernels / step	CPU launches / step	GPU busy	Step time	Idle
HuggingFace	1198	1198	15.31 ms	40.65 ms	25.34 ms (62%)
vLLM, eager	356	356	13.02 ms	25.22 ms	12.20 ms (48%)
vLLM, compile only	354	354	12.90 ms	22.41 ms	9.51 ms (42%)
vLLM, CUDA graphs	383	17	12.71 ms	12.56 ms	~0 ms (0%)

EXAMPLE 4-C: SPLIT THE WIN INTO ITS TWO CAUSES

```
total saved per token    40.65 - 12.56 = 28.09 ms

idle time removed        25.34 - 0.00 = 25.34 ms    = 90% of the win
faster device work       15.31 - 12.71 =  2.60 ms    =  9% of the win
```

The result. 90% of vLLM's batch-1 decode advantage is removed idle time, not faster kernels. HuggingFace re-enters Python 1198 times per token to launch 1198 tiny kernels. Each kernel finishes before the CPU can submit the next, so the GPU sits idle 62% of the step. This is a scheduling problem wearing a performance problem's clothes.

4.5 What CUDA graphs actually do

Look at the last row of the table again. Kernel count went *up* slightly, from 356 to 383. Busy time barely moved. What collapsed was CPU launches: 356 down to 17, because the entire decode step is recorded once as a graph and then replayed with a single call. Same kernels, same order, same compiled code. The CPU is simply no longer in the loop, and the gaps close as a consequence.

A second measured result, which was not predicted: `torch.compile` saves 2.81 ms per step when CUDA graphs are off, and 0.01 ms when they are on. The two optimisations are not additive because they attack the same term, launch overhead. Once graphs have removed it there is nothing left for compilation to remove.

4.6 Batching, in one paragraph, because it follows directly

At batch 1 the decode GEMV reads all 3.09 GB of weights to advance one sequence by one token, and the tensor cores are nearly idle. Add a second sequence and the same weight read serves two rows. The bytes do not change, the useful work doubles, and arithmetic intensity rises. So throughput climbs almost linearly while per-token latency barely moves. This is the free region. It ends when the batched GEMM is finally wide enough to saturate the tensor cores, at which point you have crossed the 142 FLOP/byte ridge, every extra sequence costs real compute, and latency starts to climb. The throughput knee in `results/vllm_batch_sweep.csv` is that crossing, measured.

4.7 Week 1 exit bar

Unaided, out loud, under 90 seconds each:

1. Whiteboard the attention formula and derive why the scores are divided by `sqrt(d_head)`.
2. State the KV cache size formula and compute 28 KB/token for this model, then say why the 2 KV heads and not the 12 query heads appear in it.
3. Show that a RoPE score depends only on `m - n`.
4. Compute the ridge point as 142 FLOP/byte and place prefill and decode either side of it.
5. Explain the 123,565-token OOM cliff, including why the measured slope was 62 KiB/token and not 28.
6. Explain the HF-to-vLLM decode gap as 90% idle time and 9% faster kernels, and say what CUDA graphs changed.

4.8 Self-test

Q1. Why does GQA shrink the KV cache but not the weights?

The cache is sized by KV heads (2 here), which GQA cuts. The weights are dominated by the MLP and the 12 query projections, which GQA does not touch. 6x smaller cache, roughly unchanged parameter count.

Q2. Softmax needs the whole row for its denominator, so how can attention ever be tiled?

Keep a running maximum and a running sum. When a new tile contains a larger maximum, rescale the accumulated result by $\exp(\text{old_max} - \text{new_max})$ and carry on. The answer is exact, not an approximation. That is online softmax, and it is what makes FlashAttention possible.

Q3. Your GPU has 51 TFLOPS. Why is your decode at 28 tok/s?

Because decode does not use FLOPS. It moves 3.09 GB of weights per token at an arithmetic intensity of exactly 1 FLOP/byte, 142x below the ridge. The binding limit is 291.5 GB/s of bandwidth, which caps the model at 94 tok/s at batch 1, and HuggingFace only reached 30% of that because the GPU was idle 62% of every step.

Q4. Walk me through a memory problem you have debugged.

The OOM sweep. Predicted 28 KiB/token, measured 62. The cause was HuggingFace's per-token `torch.cat` keeping the old and new cache live at once, which makes the real slope $2 \times 28 = 56$ KiB/token, plus attention scratch. The crash itself was fragmentation: 10,437 MiB allocated against 11,764 MiB reserved, so 1.3 GiB was free but not contiguous. PagedAttention removes both effects, and vLLM held 237,376 tokens in 6.34 GiB on the same card.

Sources: Pope et al., Efficiently Scaling Transformer Inference · kippily, Transformer Inference Arithmetic · Yu et al., Orca (continuous batching) · NVIDIA, Getting Started with CUDA Graphs

Repo sources for every measured number above: `docs/baseline-hf-results.md`, `docs/baseline-vllm-results.md`, `docs/oom-results.md`, `docs/profiling.md`, `docs/batching-results.md`, `results/profile_summary.csv`.